

Introducing My love - Operating Systems

Breeha Qasim and Bilal Wajid

In this chapter, you will

- Set up the Linux environment using Ubuntu and WSL.
- Write C programs that can take command-line inputs.
- Practice essential Linux commands.
- Write C programs that can read and write a file.
- Automate compilation using Makefiles.

Like every romance, we must first understand the one we love. We must talk, communicate, give orders, and obey. To begin our romance, we first need our love (the Linux Operating System), a love language (C in our case, and C in most cases), some loving messages (encapsulated in a shell), and its beautiful face (the terminal).

1.1 Setting Up My Love (Linux)

To set up your love, we will work with the most widely used Linux-derived distribution, Ubuntu, see Figure 1. You will either (a) need an independent Ubuntu system on your machine, or (b) run Windows Subsystem for Linux (WSL, if you are using Windows), or (c) a virtual machine. Either way, Appendix A of the book will provide you with the means to play.

Remember, you need your love to love. So, be kind to yourself, and complete the setup in Appendix A before proceeding further.

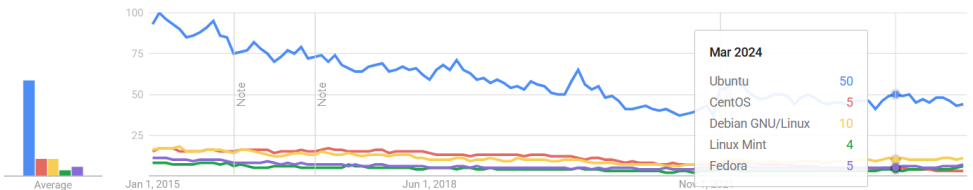


Figure 1: The figure shows a Google Trends survey (2015 – 2025) of the most popular Linux OS in the world. As you can see, Ubuntu is by far the most popular.



Linus Torvalds built Linux for fun. This fun project became one of the most influential software projects in history, powering everything from Android phones to the world's fastest supercomputers and NASA systems. Additionally, Linus also created Git, the version control system that nearly every software developer uses today, in two weeks. So, Linus changed the world twice — once with Linux, and once with Git.



Checkpoint: To complete this section, you must set up a working Ubuntu environment via native installation, or through Windows Subsystem for Linux (WSL), or through a virtual machine.

1.2 My Love Language (C)

You C, I C, and together, we all C. To communicate with your love (Linux), you must talk to her in her language (C). Hence, you must try and prepare the necessary environment to express your love.

The tools required to write your love poems (C code), you need your instruments for your melodies, these are the GNU Compiler Collection (GCC), which contains (a) the C compiler, (b) Make, and (c) GDB. These instruments help deliver system-level programs (love poems) and are included by default on Ubuntu.

To prepare yourself, read Appendix B carefully, follow it, and complete it before moving ahead. Remember, be gentle. It's your love.



The C language was created in 1972 to write the UNIX operating system. 50+ years later, we still need it.



Checkpoint: To complete this section, you must (a) establish a working C environment with gcc, make, and gdb installed; (b) create, compile, and execute a basic C program (hello.c) that prints “hello, my dear”; and (c) use a terminal-based editor (like nano or vim) to write C code.

1.3 Shell

Time to play.

Shell is your way of saying, please, my love, let's play. This makes Shell very powerful.

It all begins with the “terminal,” an interface to project your thoughts and ideas onto your love (Linux). Let's begin by learning simple commands.



The earliest Unix shells were developed in the 1970s. 50+ years later, the same commands power everything from web servers to spacecraft systems!

When you open the terminal, you'll see something like this:

```
bilal@bilal-PC:~$
```

Called the “prompt,” it waits for you to type a command. The above prompt follows a standard format

```
[username]@[hostname]:[current_directory]$
```

Here, rather than “bilal” (which is me), you will see your name and your PC’s name. (Again, I don’t mind if you name your PC and username to be “bilal” 😊). However, to avoid any confusion, for the remainder of the book, we will simply represent the prompt as the (\$) sign.

To ask your love (Linux), where am I (your current directory), ask her politely. Type the following

```
$ pwd
```

Your love will tell you where you are. I know, sometimes we forget.



Hey, why don’t you pause here and search how Linux organizes its directory structure?

Linux uses a single unified directory tree. At the top is the root directory (/), and below it are many branches (folders), which contain more folders and files. Interestingly, external drives, like USB sticks, are mounted onto the same folder structure, making navigation simpler and consistent.

Now, to move one step further, check what is contained within your present work directory by typing the following command.

```
$ ls
```

```
Desktop  Documents  Downloads  Pictures  Music  Videos
```

To get more information, typing the next command shows the following output:

```
$ ls -l
```

```
drwxr-xr-x  2 bilal bilal  4096 Jul 1 10:30 Documents
drwxr-xr-x  2 bilal bilal  4096 Jul 1 10:31 Downloads
drwxr-xr-x  2 bilal bilal  4096 Jul 1 10:32 Pictures
```

Again, you may not see my name (bilal) but your name in the output. (Again, I don’t mind you putting my name in there 😊)

Now, if you wish to see secrets hidden beneath (files starting with a dot), type the following:

```
$ ls -a
.          examples.desktop  Public
..         .gnome2          .ssh
.bash_history .gnupg                .profile
.bash_logout .ICEauthority         Templates
.bashrc      .local                .cache
.mkusb       .config          .mozilla
Desktop      Music           Documents
Pictures     Videos          Downloads
```

Now, to change directory (`cd`) to your desired location, use the following:

```
$ cd Downloads
```

This sends you to the “Downloads” folder. Check by typing “`pwd`”. Some useful ways to change the directory are shown below:

Table 1.1: `cd` shortcuts for you to try – do try them please.

Command	Description
\$ cd	The working directory is changed to the home directory. (~)
\$ cd -	The working directory is changed to the previous working directory.
\$ cd ~<user>	The working directory is changed to the home of the specified user.
\$ cd "<dir name>"	Quotes (single/double) are used with directory names containing spaces

Moreover, if you wish to see the folders/files structure displayed in a tree-like visual format, you can use:

```
$ tree
```

In the event your love doesn’t understand you, you can teach her new tricks (install new tools) by calling out the mighty “`sudo`” (Superman’s  call – super user call) using:

```
$ sudo apt install tree
```

This “`sudo`” gives you Superman’s strength, helping you (for now) teach your love new tricks and tools. Whereas `apt` helps handle packages in Ubuntu (more generally, Debian-based Linux distributions, like Ubuntu).



Let’s dive deeper. Let’s make a new folder and then delete. (Nobody should know we were here). The “make directory” (`mkdir`) and “remove directory” (`rmdir`) do just that. Try the following `mkdir` command and then check the newly made directory:

```
$ mkdir blah_blah_blah_OS
$ ls
blah_blah_blah_OS
```

Let's quickly delete this, before anyone gets to know:

```
$ rmdir blah_blah_blah_OS
```

This deletes the folder. However, the command `rmdir` only works if the folder is empty. If you try to delete a folder that contains files, you will get an error message like:

```
rmdir: failed to remove 'blah_blah_blah_OS': Directory not empty
```

So, when you need to delete a specific folder and everything inside it, use a more powerful command:

```
$ rm -rf <foldername>
```



Be careful. The `-rf` means that you `-r` recursively, and `-f` forcefully want to `rm` (remove) everything inside the designated folder `<foldername>`. Additionally, the following command contains an asterisk `*`, which is a wildcard meaning “all files” in the current directory.

```
$ rm -rf *
```

Meaning, recursively and forcefully remove all files/folders in the current directory. Always double-check before using `rm -rf *` because you cannot recover files deleted this way.

1.4 Nano

Let's learn to create a file. The easiest way is by using nano, a simple text editor that works directly inside the terminal window. To create a new file using nano, type:

```
$ nano ex.txt
```

This opens a blank screen where you can type your heart out. Let's type something that describes your ex:

Summary:

```
1 Large Envy
2 Spicy Hatred
1 Bottle of Soddness
1 Thick, guey bottle of Resentment, and
1 Parent missing
```

Special instructions: Extra crazy, please!

I know, the above is relatable. Feel free to add more. Once you've described your ex, do the following:

- Press `Ctrl + O` to save the file.

- Press Enter to confirm the filename.
- Press Ctrl + X to exit nano and return to the terminal.

You can now check that the file was created by typing:

```
$ ls
ex.txt
```

Now, let's try seeing the qualities of your ex, using different commands. Try to notice the difference between them. Try the following commands:

```
$ cat ex.txt
$ head ex.txt
$ tail ex.txt
$ head -3 ex.txt
$ tail -3 ex.txt
$ nl ex.txt
```

Now, let's copy your ex into a new file. CAUTION: You NEVER copy your ex. This only serves as an example. Again, I do need to teach, so your ex serves just as an example here. You can copy (cp) via the following:

```
$ cp ex.txt ex_2.txt
$ ls
ex.txt  ex_2.txt
```

Now you see both files. Let's open the new file and edit it. Type the following:

```
$ nano ex_2.txt
```

Inside nano, add the following line at the end:

```
NOTE: Add a slice of Evil to this order.
```

Save and exit nano (Ctrl + O → Enter → Ctrl + X). Type `tail -2 ex_2.txt` to see the updates made in your file.

If you want to move a file instead of copying it, you can use the `mv` command. This command is versatile, as it can also be used to rename files.

Let's say you want to move your ex.txt file into the Downloads folder, type:

```
$ mv ex.txt ~/Downloads/
```

This will move the ex.txt file into your Downloads folder. To rename, do the following:

```
$ mv ex_2.txt recent_ex.txt
```

This will change the file's name, keeping it in the same folder. You can verify the contents using the `cat` command.

If you ever misplace a file, like where did I place my ex, did I remember to trash it or is it still lurking somewhere, creepy right, they do that sometimes. You can use `find` to find it:

```
$ find . -name recent_ex.txt
./recent_ex.txt
```

The `(.)` dot means “search my current folder,” and `-name` specifies the file's name. If it's there, you'll find her. (Remember to trash it then. Don't leave loose ends). The `find` command is very useful when working on large projects that have many folders.

Now that you have practised the essentials of Shell and nano, let's move back to C. You C, C is important.



Checkpoint: You should now be able to (a) navigate the Linux file system, (b) create/edit new files using nano.

1.5 Pigeons for my love notes (argc & argv)

Love, as they say, is not about changing someone; it's about accepting them just the way they are. Moreover, sometimes, you want to give them something extra. A love note, a message, a whisper. This is where command-line arguments come in. You don't rewrite your code for every confession; you just pass a different message through the terminal. To send these love notes, C gives two little pigeons, (i) `int argc`, and (ii) `char * argv[]`:

```
int main(int argc, char *argv[])
```

These pigeons work together:

- `argc` (argument count): Counts the number of love notes (arguments), valued ≥ 0 .
- `argv[]` (argument vector): Holds the actual spicy content of each note. An array of character pointers. Here, `argv[0]` is always the name of your program, more like putting the recipient (the program's) name on the envelope. The real messages start from `argv[1]` and go to `argv[argc - 1]`.



Let's try writing some darling notes. Herein below, we will use these pigeons (`argc`, `argv`) to pass new arguments to the program “hello.c” allowing it to (recite) read them. How lovely. Write down the program using nano as described above.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc == 2)
        printf("I am %s. I love myself. \n", argv[1]);
    else
        printf("Usage: ./hello <your name here>. \n");
    return 0;
}
```

Program 1.1 – “hello.c”

The program “hello.c” expects to a name (argv[1]) to be used in the printf() statement.

Try compiling and running the code, as shown below:

```
$ gcc hello.c -o hello
$ hello Breeha
I am Bareeha. I love myself.
```

In the event, you don’t provide the `-o` flag with the name of the executable file (hello in this case) the compiler will produce a file `hello.out`. You can still run it using the `./` prefix (`./hello.out`) in this case, as shown below:

```
$ ./hello.out Breeha
```

Let’s spice things up. Let’s send a number of notes to a complicated, dysfunctional relationship:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("My dear: %s\n", argv[1]);

    if (argc == 5)
    {
        printf("Dear %s \n", argv[1]);
        printf("%s \n", argv[2]);
        printf("%s \n", argv[3]);
        printf("%s \n", argv[4]);
    }
    else
        printf("I expected at least a message ... Go to hell!\n");

    return 0;
}
```

Program 1.2 – “notes.c”

Compile and run:

```
$ gcc notes.c -o letter
$ letter "Jennifer" "I love you" "I hate you" "I love you"
```

You'll get this in your output:

```
Dear Jennifer
I love you
I hate you
I love you
```

Again, the above serves as an example. In the following assigned tasks, you are welcome to go crazy and ventilate yourself. (EMOTIONAL DAMAGE).

Note `argc` counts the number of arguments passed onto the program, with the name of the program (`letter`) being counted as the first argument.



AI tools are simply not allowed. If you are doing these tasks within a university laboratory environment, the faculty and teaching assistants will be watching you.

"Anything you say or do, will be used against you."

[Task 1] Print a string (a note) as many times as you want. This will require some loops. For instance, the program titled "task1" prints the given note three times:

```
task1 3 "I hate you"
I hate you
I hate you
I hate you
```



Please note that arguments passed onto a program are strings, not numbers. So, 3 above is string "3" and not number 3. To convert "3" to 3, use the function `atoi()`, of the C's standard library `<stdlib.h>` to convert string numbers like "333" to int 333. Something like `atoi(argv[1])` will work.

[Task 2] Write a program that accepts a number of strings, sorts them alphabetically, and then prints them, as shown below:

```
task2 hello my name is Bilal
Bilal
hello
is
my
name
```



You will need to use several items here. The array `argv[]` already contains the strings. So, write a simple sorting function. You learnt many in your data structures course. The `strcmp()` function can be used to compare two strings. Then write a print function for printing `argv[]`.

[Task 3] Write a program that takes two numbers and returns their sum, as shown below:

```
task3 11 5
11 + 5 = 16
```

[Task 4] Write a program that accepts multiple 5 numerical arguments, and prints its sum, minimum and maximum, as shown below:

```
Task4 11 5 20 0 9
Sum = 45
Min. = 0
Max. = 20
```



Checkpoint: Please complete all tasks 1 – 4, before proceeding further.

1.6 AAAhhhhh – My diary

If only I had a diary to write down my poems. In Linux, **files are your diary**. Your poems, in fact every poem has an opening, a middle, and an end.

- You **open** the file (hopes are high).
- You... do things (read, write, vent).
- And then... you **close it** (for everyone's sanity)

Forgetting to close your file? Wow. That's like storming off after a fight and leaving the door open, resources leak, systems crash, and your love story goes to hell. (Sweet 😊).

So how do we start? We start with:

```
FILE *fp = fopen("love_notes.txt", "mode");
```

The "mode", closely related to your *mood*, decides the kind of relationship you're entering. Are you here simply to read? Or, do you want to write as well? Or, maybe you are here, simply to put a reminder for someone's birthday, only to forget it at the right time. The following, presents modes for file operations:

Modes	Explanation
r	Read only (a file must exist)
w	Write only (creates or overwrites file)
a	Append only (writes at the end of the file)
r+	Read and write
w+	Read and write, truncates file

After opening the file, you can use functions like `fprintf()` or `fscanf()` to write or read content. When done, remember to close the file using `fclose(fp)`. Some common I/O functions are enlisted herein below:

Functions	Explanation
<code>fopen()</code>	Opens a file and returns a pointer to <code>FILE *</code>
<code>fclose()</code>	Closes a file stream
<code>fprintf()</code>	Writes formatted data to a file stream
<code>fscanf()</code>	Reads formatted data from a file stream
<code>fread()</code>	Reads binary data from a file
<code>fwrite()</code>	Writes binary data to a file
<code>feof()</code>	Checks for end-of-file indicator

Now that you’ve mastered the emotional rollercoaster of opening up, ventilating out, and finally letting go (a.k.a. file I/O), it’s time to dive into some actual relationship drama — real programs, real love letters, and raw binary heartbreak. Let’s walk through these moments, one code snippet at a time.

To begin, let’s write our first breakup letter — something light, dramatic, and slightly passive-aggressive. Note, the passive-aggressive component is a must. There is no fun without it. Remember, open the door (a.k.a. the file), pour a few bittersweet lines into it (the middle), and gently close (with `fclose()`), of course).

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("closure.txt", "w");
    if (file == NULL)
    {
        printf("Oh gosh... She blocked me... Can't open the file! \n");
        return 1;
    }
    fprintf(file, "You were right, I should've listened. \n");
    fprintf(file, "But you liked pineapple on a pizza, so we were doomed.\n");
    fclose(file);
    return 0;
}
```

Program 1.3 – “breakup.c”

The program (`breakup.c`) creates `closure.txt`, writes a couple of hard truths, and then shuts down the emotional tab. Feel free to pen your own emotional monologue or scribble down your top 3 self-love goals for this semester (because healing is hot).

Once done, let's update your diary (your very sad story):

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("closure.txt", "r");
    char line[100];

    if (file == NULL)
    {
        printf("I looked everywhere... but she's gone. \n");
        return 1;
    }

    while ( fgets(line, sizeof(line), file) )
        printf("Reading memory: %s", line);

    fclose(file);
    return 0;
}
```

Program 1.4 – “breakup.c”

This program reopens the diary, reads each line, scrolling through your chat history, wondering where it all went wrong using `fgets`.

Maybe, and you know eventually, it's time to move on. Now, let's say you're collecting data for your next relationship... Think of this as a checklist of potential suitors. Using `fprintf` and `fscanf`, let's jot down some stats about your next “*the one and only*.”

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("candidates.txt", "w");

    if (file == NULL) {
        printf("Still not ready to open up.\n");
        return 1;
    }

    fprintf(file, "Alice 20\n");
    char name[50]; int age;

    fscanf(file, "%s %d", name, &age);
    printf("Possible rebound: %s, Age: %d\n", name, age);

    fclose(file);
    return 0;
}
```

Program 1.5 – “potential_candidates.c”

Go ahead, try writing your own name and age instead — just promise you won't send it to your therapist. Once you're feeling bold, add a second entry. Yes, two *one and only(s)*. Add another `fprintf` to immortalize them, and don't forget to update your `fscanf` too — because heartbreaks are best handled in pairs, *right?*

One last thing, my love, how would you know when it's really over? *Someone as emotionally fragile as you might not even realize it.* Don't worry, the `feof()` function helps you detect this. Here's a simple example to illustrate (it's really over):

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("closure.txt", "r");
    char word[100];

    if (file == NULL) {
        printf("Closure not found. Try again later.  \n");
        return 1;
    }

    while ( !feof(file) )
    {
        fscanf(file, "%s", word);
        printf("Memory: %s\n", word);
    }

    fclose(file);
    return 0;
}
```

Program 1.6 – “file_end_of_file.c”

The `feof()` (read it as “file, end of file”) returns `true`, whenever you comes across the end of the file. Here, `!feof()` ensures you continue until you reach the end of the file.

Moreover, you can now try replacing `fscanf()` with `fgets()` to read entire lines. You can also try the above code using files with fewer or more words to observe how `feof()` helps.

*Love, like code, needs structure.
It needs an opening.
It has its middle, and
It needs its end.
It needs..., return 0;*

Let's go deeper. *Perhaps even drown. Some of you might be thinking. The guy really wants us to drown. Maybe I do. Maybe, I don't. Or maybe this is just tough love.* Nevertheless, we need to finish this.

[Task 1] Write a program (`myhead.c`) that prints the first 'n' lines of a file.

```
myhead 4 file.txt
// Reads the first four lines of the file, file.txt
```



Commands like `head`, `tail`, and `man` are called core utilities, or more broadly, Linux command-line tools or shell commands. Notice that you just made your very own OS command, equivalent to `head`. *Kudos, my dear, Kudos.*

[Task 2] Let's keep the momentum going. Write a program called "`copy.c`" that copies the contents of the first file to the second by overwriting the destination file when two filenames are supplied as arguments. (This will be the equivalent of the `cp` command in Linux).

```
copy file1.txt file2.txt
// Copies file1.txt to file2.txt
```



Checkpoint: Please complete task 1 and 2, before proceeding further. Now you can make your own commands. Good job.

1.7 Makefile: No More Repeating Yourself

Well, maybe you know this, maybe you don't. Some (normal) people, after a breakup, writing long texts in their Notes, repeatedly, (yes, this is normal), only to delete them later on, because they are not actually going to send them.

That's what compiling with long `gcc` commands feels like. Exhausting. Tedious. Emotionally draining. But fear not, **Makefile** is our CLOSURE plan.

Instead of typing out the same messy `gcc` rituals every time, you simply write your feelings (aka build instructions) once — clean, clear, and cathartic — into a `Makefile`. Then, every time you want to rebuild that project, all you need to do is whisper:

```
$ make
```

Then `make` takes care of everything, checking which files need attention, compiling what's broken, and leaving the rest alone (if only we could do that emotionally too).

So, here and now, you will dip your toes into the magical world of `Makefiles`. No drama. No complex setups. Just small, simplified examples to help you fall gently in love with the process of organized healing (a.k.a. automated compilation).

Now, before you go full Romeo writing `Makefiles` for every crush (or class project), let's be clear — this isn't the full love story. We're skipping the commitment talk: no gcc flags, no multi-file drama, no linking therapy sessions. For all that, head to Appendix C and Appendix D — your relationship coaches for `Makefiles`. They'll teach you how to build right, avoid red flags, and compile without chaos.



Checkpoint: You should now be able to (a) run a `Makefile` to compile single or multi-file programs, and (b) understand how `Make` automates the build process using written instructions.